

기술과경영 · 팀 프로젝트 설계안

학업나침반

수강 조합의 학기 부담을 정량 진단하고,
관심분야에 맞는 학기별 수강 로드맵을 제시하는 웹 서비스

대상 학부 로봇학부 · 소프트웨어학부 · 정보융합학부

팀 구성 8명 (PM 1 · 개발 3 · DB 2 · 디자인/기획 2)

배포 형태 링크 / QR 접속, 로그인 없음

작성일 2026-09-18

1. 개요

수강신청 도구는 **요일·시간이 겹치지 않는 조합**까지만 알려준다. 그러나 한 학기의 결과를 좌우하는 것은 시간표가 아니라 **과목 조합이 학기 중 언제 부담을 만드는가**이다. 본 서비스는 실제 수강한先輩들이 작성한 정량 데이터를 근거로 그 부담을 개강 전에 계산해 보여주고, 부담을 낮춘 대안 조합과 장기 수강 로드맵까지 제시한다.

두 가지 주요 기능

구분	기능 ① 학기 부담 진단 · 대안 조합	기능 ② 관심분야 수강 로드맵
시간 단위	한 학기 (16주)	남은 전 학기
입력	수강 후보 과목 조합	학부 · 학년 · 이수 과목 · 관심분야
출력	주차별 부담 그래프, 충돌 경고, 대안 조합 3안	학기별 추천 과목, 선정 사유, 진로 스토리
해결	"이번 학기를 버틸 수 있는가"	"무엇을 들어야 내 진로가 설명되는가"

2. 핵심 설계 원칙

- 원칙 1 — 횡수가 아니라 시점을 저장한다**

"과제 10회"는 판단에 쓸 수 없다. "매주 과제 + 8주차 시험 + 9주차 팀프로젝트 마감"이라야 총돌을 계산할 수 있다. 따라서 모든 부담을 **주차(1~16) 단위 이벤트**로 정규화해 저장한다. 이것이 본 서비스 데이터 모델의 핵심이며, 기존 강의평가와 구조적으로 다른 지점이다.
- 원칙 2 — 외부 권한·개인정보가 필요 없다**

학교 DB·학사 시스템에 접근하지 않는다. 데이터는 **선배의 자발적 입력**과 **공개 교과과정표**로만 구축한다. 로그인·회원가입이 없어 개인정보를 수집하지 않으며, QR 하나로 누구나 즉시 사용할 수 있다.
- 원칙 3 — 데이터 단위는 과목 × 교수 × 학기**

같은 과목이라도 담당 교수가 바뀌면 운영 방식이 달라진다. 과목명에 데이터를 묶으면 전부 오염되므로 **개설 회차 (offering)**를 기본 단위로 삼고, 교수가 바뀌면 이전 데이터의 신뢰도를 자동으로 낮춘다.
- 원칙 4 — 모르면 모른다고 표시한다**

응답 수에 따라 **신뢰도 A / B / C**를 화면에 그대로 노출한다. 데이터가 부족한 과목을 추정치로 채워 자신 있게 보여주는 것보다, "정보 부족"이라고 말하는 편이 서비스 신뢰를 지킨다.

3. 시스템 아키텍처



조합 탐색 알고리즘은 **클라이언트에서 실행** → 서버 부하 0, 응답 즉시

전 구간 무료 티어로 운영 가능하다. 서버 상시 가동이 없어 유지비가 발생하지 않으며, 학기가 끝난 뒤에도 링크가 살아 있다.

4. 데이터 모델

4.1 주차 기준선

학사일정에 따라 상수로 관리한다. 선배가 "중간고사 1주 전"이라고 답하면 내부에서 주차로 변환한다.

기준	주차	비고
학기 길이	1 ~ 16주	공휴일 보정은 하지 않음
중간고사	8주차	"중간 1주 전" = 7주차
기말고사	15 ~ 16주차	"기말 1주 전" = 14주차

4.2 스키마

```
department(id, name) -- 로봇/소프트웨어/정보융합

course(id, dept_id, code, name, credits,
       course_type, -- 전공필수 | 전공선택 | 기초
       typical_year, -- 권장 학년
       typical_term) -- 1 | 2 | 0(매학기)

offering(id, course_id, professor, year, term) -- 과목 x 교수 x 학기 ★

workload_report(
  id, offering_id, created_at,
  has_project bool,
  project_type enum, -- none | team | individual
  project_start_week int, project_due_week int,
  assignment_freq enum, -- none | weekly | biweekly | irregular | exam_only
  quiz_count int, quiz_weeks int[], -- 예: {3, 6, 11}
  exam_count int, exam_weeks int[], -- 예: {8, 15}
  presentation_weeks int[],
  subjective_load smallint) -- 1~5 체감 난도

load_event(id, offering_id, week, kind, intensity) -- 계산용 정규화 ★
-- kind : assignment | quiz | exam | project_due | presentation
-- 위 report 를 저장할 때 트리거로 자동 생성

offering_stat(offering_id, report_count, confidence) -- A>=3, B=1~2, C=0

track(id, dept_id, name) -- 제어 · 비전 · 인지/AI · 임베디드 ...
track_course(track_id, course_id, weight) -- 3=핵심 2=권장 1=보조
prerequisite(course_id, requires_id, hard) -- hard=선수 필수 / false=권장
```

왜 load_event 를 따로 두는가

workload_report 만으로는 "12주차 부담"을 구하려면 매번 배열을 펼쳐야 한다. 저장 시점에 한 번 펼쳐 두면 주차별 합산이 단순 GROUP BY week 로 끝나고, 여러 과목을 합치는 조합 탐색이 수만 회 반복돼도 즉시 계산된다. **이 정규화가 기능 ①의 성능을 결정한다.**

5. 기능 ① — 학기 부담 진단 및 대안 조합 추천

5.1 부담 점수

주차별 부담

$$W[w] = \sum (\text{event.intensity} \times \text{kind_weight})$$

담은 과목의 w주차 이벤트 전체

w = 1..16

kind_weight

exam 3.0 | project_due 2.5 | presentation 2.0
quiz 1.2 | assignment 1.0

(초기값, 검증 후 보정)

학기 점수

$$\text{Score} = 0.5 \times \text{mean}(W) + 0.5 \times \text{top3_mean}(W)$$

총량만 보면 "부담이 한 시기에 몰린 시간표"와 "고르게 퍼진 시간표"가 같은 점수를 받는다. 상위 3개 주차 평균을 절반 반영해 몰림을 점수에 직접 반영한다.

점수는 내부 계산에만 쓰고 화면에는 단계와 경고 문장으로만 노출한다. "87점"보다 "이 과목을 빼면 한 단계 내려갑니다"가 행동으로 이어진다.

5.2 충돌 규칙 — 규칙 기반이므로 근거를 설명할 수 있다

규칙	조건	출력 문구
R1	같은 주차에 project_due 2건 이상	"N주차 프로젝트 마감 중복"
R2	시험 주차 ±1 에 project_due 존재	"시험기간에 프로젝트 마감이 겹칩니다"
R3	assignment_freq = weekly 과목 2개 이상	"매주 과제 과목이 N개 — 상시 누적"
R4	$W[w] > \text{threshold}$	"N주차 과부하 — 최대 고비"
R5	팀 프로젝트 과목 3개 이상	"팀 프로젝트 3개 동시 — 일정 조율 부담"

5.3 대안 조합 탐색

입력

후보 과목 N개 (보통 12~20), 목표 과목 수 k (5~7)
사용자가 "반드시 듣는 과목"을 lock 가능

1) 조합 생성

$C(N, k)$

N=20, k=6 → 38,760 가지

2) 필터

시간표 충돌 제거 · 이수구분 제약(전필/전선 최소 학점)

3) 평가

각 조합의 Score 계산 (load_event 합산)

4) 정렬 · 제시

상위 3개 + 현재 조합 대비 개선 폭 표시

※ 후보군이 커지면 ($C(N,k) > 200,000$) greedy + local search 로 자동 전환

"불가피하게 들어야 하는 과목"에 대한 대응

전공필수처럼 뺄 수 없는 과목은 lock 해 두고 나머지만 탐색한다. 그래도 위험 주차가 남으면 대안 제시 대신 사전 대비 안내로 전환한다 — "10주차에 과제 3건과 시험 2건이 겹칩니다. 해당 프로젝트는 7주차까지 초안을 마쳐 두는 것을 권장합니다." 이것이 개강 전에 미리 대비한다는 본 서비스의 목적에 직접 대응하는 출력이다.

6. 기능 ② — 관심분야 기반 학기별 수강 로드맵

6.1 문제

로봇학부만 해도 제어·비전·인지/AI·임베디드 등 분야가 나뉘고 각 분야의 전공이 개설되지만, **전부 들을 수는 없다**. 무엇을 골라야 취업이나 대학원 진학 때 **하나의 스토리로 설명되는 이력**이 되는지에 대한 안내가 없다.

6.2 처리 절차

입력 학부 · 현재 학년/학기 · 이수한 과목(체크) · 관심 트랙 1~2개

- 1) 과목 점수화 $score(c) = \sum track_course.weight$ (선택한 트랙 기준)
- 2) 수강 가능 판정 prerequisite 그래프 위상정렬 → 선수과목 미이수 과목 제외
- 3) 학기 배치 typical_term(개설 학기) 제약 + 학기당 학점 상한(18)
- 4) 학기 내 평탄화 기능 ①의 Score 로 같은 학기 조합의 부담을 최소화
- 5) 사유 생성 "제어 트랙 핵심 2과목 확보 · 임베디드 실습 기반 마련"

6.3 출력 예시

관심분야 : 제어 + 임베디드

현재 : 2학년 1학기 진입

2-1	제어 트랙 진입 기반	선형대수 · 회로이론 · 프로그래밍
2-2	이론 축 확보	신호및시스템 · 마이크로프로세서
3-1	★ 트랙 핵심	제어공학 · 임베디드시스템
3-2	실습 · 응용	로봇공학 · 임베디드SW실습
4-1	통합	캡스톤설계 (제어 주제)

CV 스토리 : "제어 이론 → 임베디드 구현 → 캡스톤에서 실기 검증"
면접에서 한 문장으로 설명되는 수강 이력

트랙별 과목 매핑은 DB 구축 담당이 각 학부 교과과정표를 보고 작성한다. 트랙 정의와 가중치는 교수·선배 검토를 거쳐 확정한다.

7. 선배 입력 폼 설계 — 3분 안에 끝나야 한다

데이터가 모이지 않으면 서비스가 성립하지 않는다. **기여 경험 설계가 개발만큼 중요하다**.

#	문항	입력 방식
1	과목 · 담당교수 · 수강학기	검색 후 선택 (자동완성)
2	프로젝트가 있었나요?	없음 / 팀 / 개인 3버튼
3	프로젝트 마감일이 언제였나요?	"중간 무렵 · 기말 1주 전 · 종강 주" 버튼 → 주차로 변환
4	과제 주기	매주 / 격주 / 비정기 / 중간·기말만 / 없음
5	퀴즈가 있었나요? 언제?	유무 + 16주 체크박스 그리드
6	시험 횟수와 시기	중간 / 기말 / 추가시험 체크
7	체감 난도	1~5 별점

- 주차는 **절대 텍스트로 입력받지 않는다**. 상대 표현 버튼과 체크박스 그리드만 사용한다.
- 로그인·회원가입 없음. 익명 제출, 중복은 브라우저 토큰으로만 완화.
- 제출 직후 **해당 과목의 집계 결과를 바로 보여준다** — 기여에 대한 즉각적 보상.

8. 기술 스택

영역	선택	근거
프론트엔드	Next.js (App Router) · TypeScript · Tailwind	페이지와 API를 한 레포에서 관리, Vercel 배포가 즉시
시각화	자체 SVG (막대 · 히트맵)	차트 라이브러리 없이 충분. 번들 경량, 디자인 자유
API	Next.js Route Handlers	서버리스 — 상시 가동 비용 없음
DB	Supabase PostgreSQL	무료 티어, 배열 타입·트리거 지원, 관리 화면 제공
배포 · 접속	Vercel + QR 코드	포스터·단톡방에 QR만 배포하면 누구나 즉시 사용
조합 탐색	클라이언트 실행 (Web Worker)	서버 부하 0, UI 멈춤 없음
협업	GitHub · Notion · Figma	이슈로 담당자·기한 관리

9. 역할 분담

역할	인원	담당 업무
PM (총괄)	1	일정·범위 관리, 교수님·이해관계자 소통, 주간 회의 운영, 통합 브랜치 관리, 최종 의사결정
개발	3	FE-A 과목 담기·조합 비교·입력 폼 / FE-B 주차별 부담 시각화·로드맵 화면 / BE API·스키마 구현·부담 계산 엔진·조합 탐색 알고리즘
DB 구축	2	3개 학부 교과과정표 수집 → 과목 마스터 구축, 선수과목 관계 정리, 트랙 매핑 작성, 선배 응답 수집 캠페인 운영 , 데이터 검증
디자인·기획	2	요구사항 정의, 와이어프레임, UI 디자인, 입력 폼 문항 설계, 사용자 테스트 설계·진행, 발표 자료

10. 주차별 계획 (11주 · 최종 발표 12월 첫째 주 가정)

주차	마일스톤	개발 (3)	DB (2)	디자인·기획 (2)	PM (1)
W1 9/22	kickoff	레포·환경 세팅, 기술 스택 확정	3개 학부 교과과정표 수집 착수	요구사항 정의서 초안	범위 동결, 역할 배정, 회의 규칙
W2	설계 확정	API 계약 정의, 스키마 v1	과목 마스터 입력 시작	와이어프레임, 입력 폼 문항 확정	일정·리스크 관리표
W3	골격	FE 스켈레톤, DB 연결	과목 마스터 1차 완료	UI 시안 · 컬러·타이포	중간 점검 기준 합의
W4 ★	입력 폼 배포	입력 폼 + POST API 완성, 배포	선배 응답 수집 캠페인 시작	QR·포스터 제작, 배포 채널 확보	학과 단독·학회 협조 요청
W5	중간 점검	부담 계산 엔진, 주차별 타임라인	선수과목 관계 정리	진단 화면 디자인	중간 보고
W6	규칙·탐색	충돌 규칙 R1~R5, 조합 탐색	응답 1차 검증·정제	경고 문구 작성	진척 점검, 병목 해소
W7	MVP 완성	대안 추천 UI 관통	트랙 정의·매핑 착수	사용성 1차 점검	MVP 리뷰
W8	기능 ②	로드맵 알고리즘 구현	트랙 매핑 완료, 교수·선배 검토	로드맵 화면 디자인	범위 재확인 (축소 판단)
W9	통합	로드맵 UI, 신뢰도 표시	데이터 2차 검증	사용자 테스트 설계	발표 구성 초안
W10	검증	버그 수정, 성능	소급 검증 데이터 대조	사용자 테스트 20명	피드백 정리·우선순위
W11	최종 발표	안정화, 배포 점검	최종 데이터 확정	발표 자료·리허설	최종 보고서 제출

W4가 이 프로젝트의 성패를 가른다

모든 기능은 데이터가 있어야 작동한다. 따라서 **진단 기능보다 입력 폼을 먼저 배포**해 선배 응답 수집 기간을 최대한 확보한다. 개발이 끝난 뒤 데이터를 모으기 시작하면 학기가 끝난다. 수집 기간을 **7주 이상** 확보하는 것이 목표다.

11. 측정 가능한 목표

항목	목표치
과목 마스터 구축	3개 학부 전공과목 150과목 이상 (과목명·학수번호·선수관계·개설학기)
부담 데이터 확보	80개 개설 회차 이상 , 그중 신뢰도 A(응답 3건 이상) 30개 이상
응답 수집	선배 응답 150건 이상
응답 소요 시간	과목 1개당 3분 이내
진단 응답 속도	조합 입력 → 결과 출력 3초 이내
정확도 (소급 검증)	지난 학기 수강생 15명에게 "가장 힘들었던 주차"를 묻고 대조 → 70% 이상 일치
사용자 테스트	2학년 이상 20명 , "다음 수강신청에 쓰겠다" 70% 이상

12. 위험 및 대응

위험	대응
선배 응답이 안 모인다 (최대 리스크)	팀원 8명이 각자 수강한 과목을 먼저 입력해 초기 데이터를 확보 한 뒤 공개한다. 빈 서비스는 아무도 쓰지 않는다. 학회·단톡방·과방 QR 포스터로 배포하고, 응답 시 해당 과목 결과를 즉시 보여줘 기여 동기를 만든다.
교수가 바뀌어 데이터가 무효	offering 단위 저장. 교수 변경 시 신뢰도 자동 강등, 화면에 "2025-2학기 · ○○○ 교수 기준" 병기
응답 신뢰성 (허위·편향)	중앙값 집계로 이상치 완화, 응답 3건 미만은 신뢰도 C 표시. 교수 개인에 대한 평가·점수는 기능에서 제외
범위 과다 (기능 2개 + 3개 학부)	W8에 범위 재확인 게이트. 지연 시 기능 ②를 로봇학부만 으로 축소하고 나머지는 향후 과제로 발표
교과과정 데이터 수집 지연	W3까지 과목 마스터 1차 완료를 고정 마일스톤으로 관리. 미달 시 대상 학부를 2개로 축소
알고리즘 성능	조합 폭발 시 greedy + local search 자동 전환. Web Worker로 UI 블로킹 방지

13. 범위 밖 (명시적으로 하지 않는 것)

- 학교 학사 시스템 연동, 수강신청 대행·매크로
- 개인정보 수집, 회원가입, 성적 입력
- **교수 개인에 대한 평가·랭킹·점수화** — 과목의 운영 방식만 다룬다
- 졸업요건 판정 (학교 포털이 이미 제공)
- 모바일 네이티브 앱 — 반응형 웹으로 대체

본 문서는 설계 계획안이며, 데이터 수치와 주차 배치는 팀 논의 및 학사일정 확정 후 조정한다.